

TASK# 1

1. Updates the APT package index and upgrades all installed packages.
`sudo apt update && sudo apt upgrade -y`
2. Reboots the system to apply kernel updates.
`sudo reboot`
3. Disables swap memory temporarily.
`sudo swapoff -a`
4. Disables swap permanently by editing the fstab file.
`sudo sed -i 's/ swap / s/^/#/' /etc/fstab`
5. Verifies that swap is disabled.
`swapon --show`
6. Loads the overlay filesystem kernel module.
`sudo modprobe overlay`
7. Loads the bridge netfilter kernel module.
`sudo modprobe br_netfilter`
8. Configures kernel modules to load automatically on boot.
`sudo tee /etc/modules-load.d/k8s.conf << EOF`
`overlay`
`br_netfilter`
`EOF`
9. Configures required Kubernetes networking kernel parameters.
`sudo tee /etc/sysctl.d/k8s.conf << EOF`
`net.bridge.bridge-nf-call-iptables = 1`
`net.bridge.bridge-nf-call-ip6tables = 1`
`net.ipv4.ip_forward = 1`
`EOF`
10. Applies all kernel parameter changes.
`sudo sysctl --system`
11. Installs the containerd container runtime.
`sudo apt install -y containerd`
12. Creates the containerd configuration directory.
`sudo mkdir -p /etc/containerd`
13. Generates the default containerd configuration file.
`sudo containerd config default | sudo tee /etc/containerd/config.toml`
14. Configures containerd to use systemd cgroups.
`sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'`
`/etc/containerd/config.toml`
15. Restarts the containerd service.
`sudo systemctl restart containerd`
16. Enables containerd to start on system boot.
`sudo systemctl enable containerd`

17. Updates the package index before adding Kubernetes repository.
`sudo apt-get update`
18. Installs required packages for secure repository access.
`sudo apt-get install -y apt-transport-https ca-certificates curl gpg`
19. Creates a directory for Kubernetes APT keyrings.
`sudo mkdir -p /etc/apt/keyrings`
20. Downloads and installs the Kubernetes repository signing key.
`curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg`
21. Adds the Kubernetes v1.32 APT repository.
`echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list`
22. Refreshes the package index after adding Kubernetes repository.
`sudo apt-get update`
23. Installs Kubernetes components kubelet, kubeadm, and kubectl.
`sudo apt-get install -y kubelet=1.32.* kubeadm=1.32.* kubectl=1.32.*`
24. Prevents Kubernetes packages from being automatically upgraded.
`sudo apt-mark hold kubelet kubeadm kubectl`
25. Sets a unique hostname for the Kubernetes node.
`sudo hostnamectl set-hostname "k8s-master01"`
26. Reloads the shell to apply hostname changes.
`exec bash`
27. Initializes the Kubernetes control-plane node.
`sudo kubeadm init --pod-network-cidr=10.244.0.0/16`
28. Creates the Kubernetes configuration directory for kubectl.
`mkdir -p $HOME/.kube`
29. Copies the admin kubeconfig file for cluster access.
`sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config`
30. Changes ownership of the kubeconfig file to the current user.
`sudo chown $(id -u):$(id -g) $HOME/.kube/config`
31. Installs the Calico pod network plugin.
`kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.29.1/manifests/calico.yaml`
32. Enables the UFW firewall.
`sudo ufw enable`
33. Allows SSH access through the firewall.
`sudo ufw allow ssh`
34. Allows Kubernetes API server traffic through the firewall.
`sudo ufw allow 6443/tcp`
35. Allows etcd communication ports.
`sudo ufw allow 2379:2380/tcp`

36. Allows kubelet API communication.
sudo ufw allow 10250/tcp
37. Allows kube-controller-manager communication.
sudo ufw allow 10257/tcp
38. Allows kube-scheduler communication.
sudo ufw allow 10259/tcp
39. Allows Calico VXLAN networking traffic.
sudo ufw allow 4789/udp
40. Allows Calico BGP traffic.
sudo ufw allow 179/tcp
41. Allows pod network CIDR traffic.
sudo ufw allow from 10.244.0.0/16
42. Removes control-plane taint to allow workloads on a single node.
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
43. Displays the status of Kubernetes nodes.
kubectl get nodes
44. Lists all pods across all namespaces.
kubectl get pods -A
45. Displays Kubernetes cluster information.
kubectl cluster-info
46. Shows Kubernetes client and server versions.
kubectl version
47. Displays the installed kubeadm version.
kubeadm version

```
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

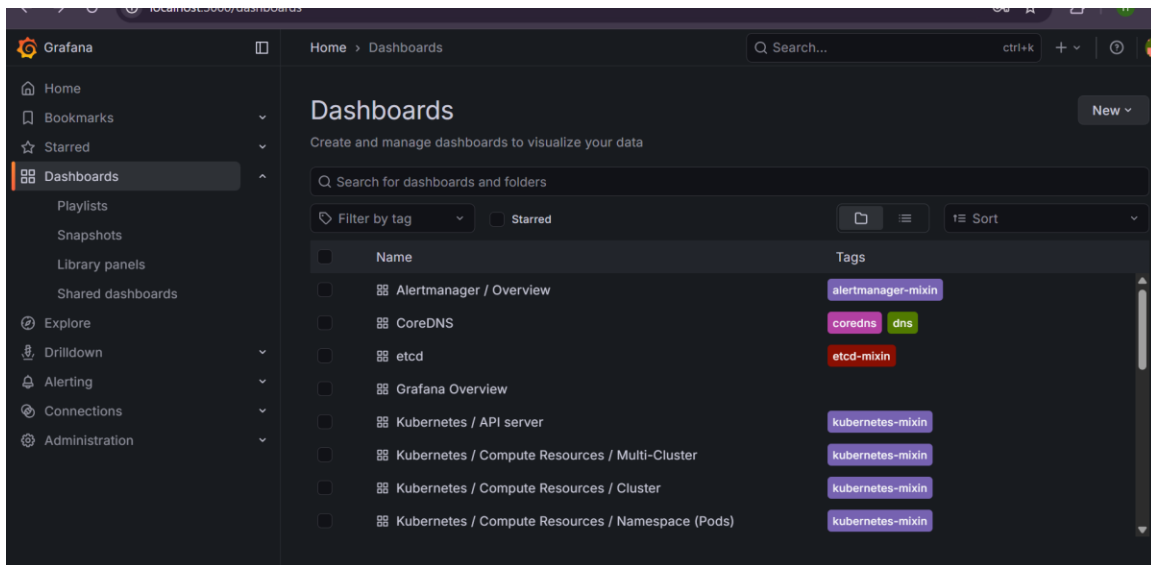
Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 172.29.254.201:6443 --token 6j3mds.rqo2hignxe3aax18 \
--discovery-token-ca-cert-hash sha256:cbc4daa73c8d9ca81a2303c7bdf2033a0185fdee1cf3e95f0b323fb54e663ec3
namespace/kube-flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
serviceaccount/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
wais_ali@k8s-master01:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/Assingment4/Task 1$ kubectl version --short
error: unknown flag: --short
See 'kubectl version --help' for usage.
wais_ali@k8s-master01:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/Assingment4/Task 1$ kubectl version
Client Version: v1.32.10
Kustomize Version: v5.5.0
Server Version: v1.32.10
```

TASK#2

1. Downloads and installs Helm package manager.
`curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash`
2. Verifies Helm installation and version.
`helm version`
3. Adds the NGINX Ingress Controller Helm repository.
`helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx`
4. Updates all Helm repositories.
`helm repo update`
5. Installs NGINX Ingress Controller as a DaemonSet in the ingress-nginx namespace.
`helm install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-nginx --create-namespace --set controller.kind=DaemonSet --set controller.hostNetwork=false --set controller.service.type=ClusterIP`
6. Displays the status of NGINX Ingress Controller pods.
`kubectl get pods -n ingress-nginx`
7. Adds the Metrics Server Helm repository.
`helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/`
8. Updates Helm repositories after adding Metrics Server.
`helm repo update`
9. Installs Metrics Server in the kube-system namespace.
`helm install metrics-server metrics-server/metrics-server -n kube-system --set args[0]=--kubelet-insecure-tls --set args[1]=--kubelet-preferred-address-types=InternalIP,Hostname`
10. Displays CPU and memory usage of all nodes.
`kubectl top nodes`
11. Displays CPU and memory usage of all pods in all namespaces.
`kubectl top pods -A`
12. Adds the Prometheus Community Helm repository.
`helm repo add prometheus-community https://prometheus-community.github.io/helm-charts`
13. Updates Helm repositories after adding Prometheus Community.
`helm repo update`
14. Installs Prometheus, Alertmanager, and Grafana monitoring stack.
`helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace --set prometheusOperator.admissionWebhooks.enabled=false --set prometheusOperator.resources.requests.memory=200Mi --set prometheusOperator.resources.limits.memory=300Mi --set prometheus.resources.requests.memory=300Mi --set alertmanager.resources.requests.memory=200Mi --set nodeExporter.enabled=false`

15. Displays the status of monitoring pods.
kubecttl get pods -n monitoring
16. Retrieves the Grafana admin password from Kubernetes secret.
kubecttl get secret monitoring-grafana -n monitoring -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
17. Forwards Grafana service to local port 3000 for browser access.
kubecttl port-forward svc/monitoring-grafana -n monitoring 3000:80
18. Verifies monitoring pods are running correctly.
kubecttl get pods -n monitoring
19. Verifies node resource usage using Metrics Server.
kubecttl top nodes
20. Verifies pod resource usage across all namespaces.
kubecttl top pods -A



TASK#3

1. Deploys the Redis Leader deployment.
`kubectl apply -f https://k8s.io/examples/application/guestbook/redis-leader-deployment.yaml`
2. Shows pods with app=redis and role=leader.
`kubectl get pods -l app=redis -l role=leader`
3. Checks logs of the Redis Leader deployment (optional).
`kubectl logs -f deployment/redis-leader`
4. Exposes Redis Leader via a Kubernetes Service.
`kubectl apply -f https://k8s.io/examples/application/guestbook/redis-leader-service.yaml`
5. Displays the Redis Leader service.
`kubectl get service redis-leader`
6. Deploys Redis Follower deployment.
`kubectl apply -f https://k8s.io/examples/application/guestbook/redis-follower-deployment.yaml`
7. Shows pods with app=redis and role=follower.
`kubectl get pods -l app=redis -l role=follower`

8. Exposes Redis Followers via a Kubernetes Service.
kubectl apply -f <https://k8s.io/examples/application/guestbook/redis-follower-service.yaml>
9. Displays the Redis Follower service.
kubectl get service redis-follower
10. Deploys Guestbook Frontend deployment.
kubectl apply -f <https://k8s.io/examples/application/guestbook/frontend-deployment.yaml>
11. Shows pods with app=guestbook and tier=frontend.
kubectl get pods -l app=guestbook -l tier=frontend
12. Exposes Guestbook Frontend via a Kubernetes Service.
kubectl apply -f <https://k8s.io/examples/application/guestbook/frontend-service.yaml>
13. Displays the Frontend service.
kubectl get service frontend
14. Port-forwards the frontend service to local port 8080.
kubectl port-forward svc/frontend 8080:80
15. Scales the Frontend deployment up to 5 replicas.
kubectl scale deployment frontend --replicas=5
16. Shows frontend pods after scaling.
kubectl get pods -l app=guestbook -l tier=frontend
17. Scales the Frontend deployment down to 2 replicas.
kubectl scale deployment frontend --replicas=2
18. Shows frontend pods after scaling down.
kubectl get pods -l app=guestbook -l tier=frontend
19. Creates
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: guestbook-ingress
namespace: default
annotations:
nginx.ingress.kubernetes.io/rewrite-target: /
spec:
rules:

- host: guestbook.local
http:
paths:
 - path: /
pathType: Prefix
backend:
service:
name: frontend
port:
number: 80
EOF

20. Displays the Guestbook ingress.

kubectl get ingress guestbook-ingress

21. Adds host mapping for local testing (edit hosts file).

sudo nano /etc/hosts

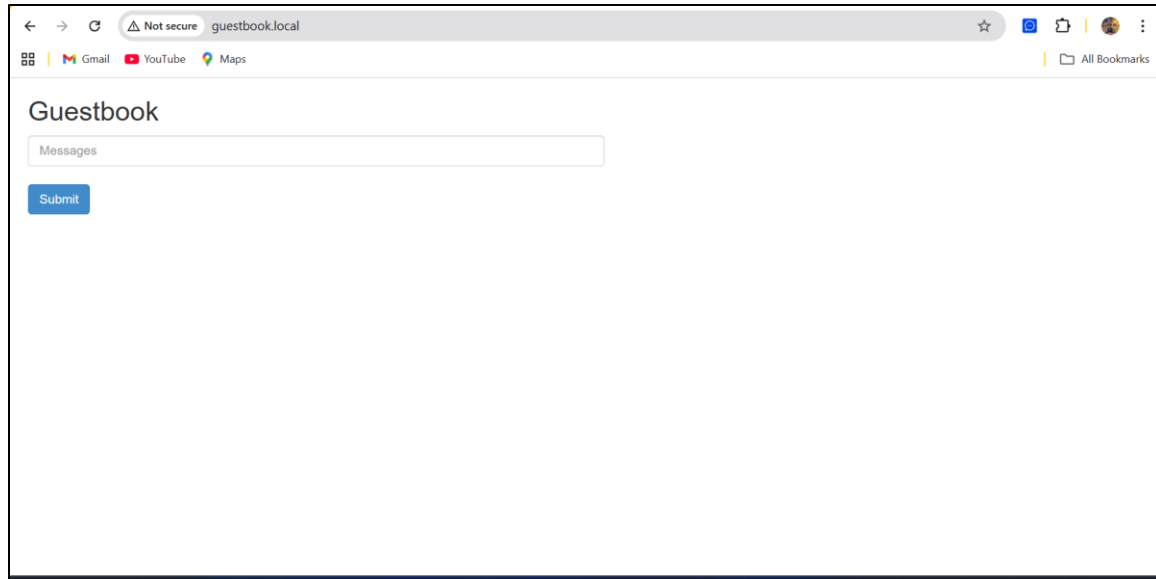
22. Adds line for Guestbook ingress to hosts file.

127.0.0.1 guestbook.local

23. Tests Guestbook frontend in browser.

<http://guestbook.local>

```
wais_ali@LAPTOP-LGLSHIK6: ~$ kubectl get pods -l app=redis -l role=leader
NAME                                READY    STATUS    RESTARTS    AGE
redis-leader-5f4cc9b47d-vscvv       1/1      Running   6 (13m ago)  8d
wais_ali@LAPTOP-LGLSHIK6: ~$ kubectl get service redis-leader
NAME    TYPE    CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
redis-leader ClusterIP  10.99.107.9    <none>         6379/TCP   8d
wais_ali@LAPTOP-LGLSHIK6: ~$ kubectl get pods -l app=redis -l role=follower
No resources found in default namespace.
wais_ali@LAPTOP-LGLSHIK6: ~$ kubectl get service redis-follower
NAME    TYPE    CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
redis-follower ClusterIP  10.106.42.117 <none>         6379/TCP   7d2h
wais_ali@LAPTOP-LGLSHIK6: ~$ kubectl get pods -l app=guestbook -l tier=frontend
NAME                                READY    STATUS    RESTARTS    AGE
frontend-7c457c988c-7jpb2          1/1      Running   6 (14m ago)  8d
frontend-7c457c988c-hxpn7          1/1      Running   6 (14m ago)  8d
frontend-7c457c988c-zzgvw          1/1      Running   6 (14m ago)  8d
wais_ali@LAPTOP-LGLSHIK6: ~$ kubectl get service frontend
NAME    TYPE    CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
frontend ClusterIP  10.111.240.100 <none>         80/TCP     8d
wais_ali@LAPTOP-LGLSHIK6: ~$
```

TASK#4

1. Removes existing Kubernetes APT repository.
`sudo rm -f /etc/apt/sources.list.d/kubernetes.list`
2. Adds Kubernetes v1.33 APT repository.
`echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core/stable/v1.33/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list`
3. Updates package index after adding new repository.
`sudo apt update`
4. Unholds kubeadm package to allow upgrade on control plane.
`sudo apt-mark unhold kubeadm`
5. Installs kubeadm v1.33 on control plane node.
`sudo apt install -y kubeadm=1.33.*`
6. Holds kubeadm package to prevent automatic upgrades.
`sudo apt-mark hold kubeadm`
7. Verifies installed kubeadm version.
`kubeadm version`
8. Checks the Kubernetes upgrade plan.
`sudo kubeadm upgrade plan`

9. Applies upgrade on control plane node (replace x with latest patch).
`sudo kubeadm upgrade apply v1.33.x`
10. Unholds kubelet and kubectl to allow upgrade on control plane.
`sudo apt-mark unhold kubelet kubectl`
11. Installs kubelet and kubectl v1.33 on control plane node.
`sudo apt install -y kubelet=1.33.* kubectl=1.33.*`
12. Holds kubelet and kubectl to prevent automatic upgrades.
`sudo apt-mark hold kubelet kubectl`
13. Reloads systemd manager configuration.
`sudo systemctl daemon-reload`
14. Re-executes systemd manager.
`sudo systemctl daemon-reexec`
15. Restarts kubelet service.
`sudo systemctl restart kubelet`
16. Verifies Kubernetes client and server versions.
`kubectl version`
17. Displays status of all nodes.
`kubectl get nodes`
18. Removes Kubernetes APT repository on each worker node before upgrade.
`sudo rm -f /etc/apt/sources.list.d/kubernetes.list`
19. Adds Kubernetes v1.33 APT repository on each worker node.
`echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.33/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list`
20. Updates package index on each worker node.
`sudo apt update`
21. Unholds kubeadm package on worker node for upgrade.
`sudo apt-mark unhold kubeadm`
22. Installs kubeadm v1.33 on worker node.
`sudo apt install -y kubeadm=1.33.*`
23. Holds kubeadm to prevent automatic upgrades.
`sudo apt-mark hold kubeadm`

24. Upgrades the worker node to match control plane version.
sudo kubeadm upgrade node
25. Unholds kubelet and kubectl on worker node.
sudo apt-mark unhold kubelet kubectl
26. Installs kubelet and kubectl v1.33 on worker node.
sudo apt install -y kubelet=1.33.* kubectl=1.33.*
27. Holds kubelet and kubectl to prevent automatic upgrades.
sudo apt-mark hold kubelet kubectl
28. Reloads systemd manager configuration on worker node.
sudo systemctl daemon-reload
29. Re-executes systemd manager on worker node.
sudo systemctl daemon-reexec
30. Restarts kubelet service on worker node.
sudo systemctl restart kubelet
31. Verifies all nodes after upgrade.
kubectl get nodes
32. Displays all pods across all namespaces to confirm cluster health.
kubectl get pods -A

```
kubectl get pods -A
NAME                                STATUS    ROLES    AGE   VERSION
laptop-lglshik6                    Ready     control-plane  40h   v1.33.7

NAMESPACE   NAME                                READY    STATUS    RESTARTS   AGE
default      frontend-7c457c988c-7jq2           1/1      Running   0           97m
default      frontend-7c457c988c-hxpn7          1/1      Running   0           97m
default      frontend-7c457c988c-zzgvw          1/1      Running   0           97m
default      redis-follower-854f6dcdb5-2k8hc     1/1      Running   0          106m
default      redis-follower-854f6dcdb5-mv47z     1/1      Running   0          106m
default      redis-leader-5f4cc9b47d-vscvv       1/1      Running   0          125m
grafana      grafana-788dd8c864-dfpvs            1/1      Running   1 (3h29m ago)  24h
ingress-nginx ingress-nginx-controller-cj48w        1/1      Running   2 (3h29m ago)  29h
kube-flannel kube-flannel-ds-l49cw               1/1      Running   3 (3h29m ago)  40h
kube-system  coredns-674b8bbfcf-bkzx2            1/1      Running   0          3m33s
kube-system  coredns-674b8bbfcf-bw6b6            1/1      Running   0          3m33s
kube-system  etcd-laptop-lglshik6                1/1      Running   1 (2m16s ago)  2m12s
kube-system  kube-apiserver-laptop-lglshik6       1/1      Running   1 (2m16s ago)  2m12s
kube-system  kube-controller-manager-laptop-lglshik6 1/1      Running   1 (2m16s ago)  2m12s
kube-system  kube-proxy-47qv7                    1/1      Running   0          3m33s
kube-system  kube-scheduler-laptop-lglshik6       1/1      Running   0          2m12s
kube-system  metrics-server-5dd7b49d79-6wvvp      1/1      Running   2 (3h29m ago)  29h
monitoring  alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2      Running   2 (3h29m ago)  23h
monitoring  monitoring-grafana-66fc65d6b9-hncxb  3/3      Running   0          3h15m
monitoring  monitoring-kube-prometheus-operator-595ff7875b-kbgvc    1/1      Running   1 (3h29m ago)  23h
monitoring  monitoring-kube-state-metrics-7f575674-zptjz            1/1      Running   3 (4m23s ago)  23h
monitoring  prometheus-monitoring-kube-prometheus-prometheus-0      2/2      Running   2 (3h29m ago)  23h
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/Assingment4/Task 3/aplication$
kubectl version
Client Version: v1.33.7
Kustomize Version: v5.6.0
Server Version: v1.33.7
wais_ali@LAPTOP-LGLSHIK6:/mnt/c/Users/GHALI/OneDrive - Higher Education Commission/8th semester/devops/Assingment4/Task 3/aplication$
```